

BINGKASAN HASIL ALGORITMA PENJADWALAN PROSES		
Istilah	Definisi & Rumus	
Waiting Time (WT)	WT = Selesai Eksekusi – Waktu Tiba – Lama Eksekusi	
Turn Around Time (TAT)	TAT = WT + Lama Eksekusi	
AWT / ATAT (rata-rata)	$AWT = \Sigma WT / n$ $ATAT = \Sigma TAT / n$	
Scheduler	Memilih proses dari ready queue yang akan dieksekusi	
Dispatcher	Rutin OS yang mengalihkan eksekusi ke proses yang dipilih scheduler	

Type / Strategi	Keterangan	Contoh Algoritma
Jangka Pendek	Memilih proses dari ready queue (paling sering dilakukan)	FIFO, SJF, RR, PS, GS
Jangka Menengah	Swapping proses ke memori sekunder jika memori penuh	OS modern (Virtual Mem)
Jangka Panjang	Memilih program yang akan masuk ke memori utama & ready queue	Batch system
Non-Preemptive	CPU tidak bisa diambil alih, proses jalan sampai selesai	FIFO, SJF, HRN

• Urut berdasarkan waktu kedatangan (arrival time)

• **WT = Mulai Eksek - Waktu Tiba**

• **ATAT = WT + Lama Eksekusi**

• Pilih proses dengan Rasio Penalti TERTINGGI

• **Sia-sia = Sls Eksek sebelumnya - Waktu Tiba**

• **Ratio = (Sia-sia + BT) / BT**

C. ALGORITMA PENJADWALAN — RUMUS & CONTOH PERHITUNGAN

P	Tiba	BT	Mulai	Selesai	WT	TAT
A	0	5	0	5	0	5
B	0	2	5	7	5	7
C	0	6	7	13	7	13
D	0	8	13	21	13	21
E	0	3	21	24	21	24
					AWT=9.2	ATAT=14

• Burst Time terkecil dieksekusi duluan → AWT optimal

• **SJF - Shortest Job First (Non-Preemptive)**

• Rumus sama dengan SJF, urutan berdasarkan BT, mulai

P	Tiba	BT	Mulai	Selesai	WT	TAT
A	0	1	0	1	0	1
C	5	3	5	8	0	3
B	2	5	8	13	6	11
E	9	7	13	20	4	11
D	7	10	20	30	13	23
					AWT=4.6	ATAT=9.8

• Prioritas tertinggi (angka terkecil) dieksekusi lebih dulu

• **3. Priority Scheduling - PS (Pre & Non-Pre)**

• Starvation & solusi: Aging (naikkan prioritas tiap waktu tertentu)

• **WT = Mulai Eksek - Waktu Tiba | TAT = WT + BT**

P	BT	Prioritas	Urutan	WT	TAT
P2	1	1	1st	0	1
P5	5	2	2nd	1	6
P1	10	3	3rd	6	16
P3	2	3	4th	16	18
P4	1	4	5th	18	19
					AWT=8.2 ATAT=12

Algoritma	Jenis	Kelebihan	Kekurangan
FIFO/FCFS	Non-Preemptive	Sederhana, mudah diimplementasi, adil secara urutan	Response time buruk, convoy effect jika proses panjang ad
SJF	Non-Preemptive	AWT/Waiting Time paling optimal (minimum)	Butuh prediksi BT sebelumnya, proses panjang bisa starvat
HRN	Non-Preemptive	Mencegah starvation, responsif terhadap lama tunggu	Overhead hitung rasio tiap ronde, sensitif urutan kedatan
Round Robin	Preemptive	Fair untuk semua proses, cocok time-sharing, responsif	Banyak context switch jika Q kecil; ATAT besar; Q harus t
Priority	Pre & Non-Pre	Proses penting/kritis didahulukan sesuai kepentingan	Starvation untuk prioritas rendah (solusi: aging)
Guaranteed	Preemptive	CPU terbagi rata/proporsional, adil untuk semua user	Kompleks, overhead perhitungan JWP & rasio, tidak umum

P	Tiba	BT	Mulai	Sls	RT
A	0	4	0	4	4
B(2.5)	1	2	4	6	5
C(1.8)	2	5	6	11	9
E(2.75)	4	4	11	15	11
D(2.0)	3	8	15	23	20
					ATAT=9.8

• **Ratio dihitung**

• Setiap proses diberi Quantum (Q) waktu CPU secara bergilir

• **Round Robin - RR (Preemptive)**

• Q harus < waktu selesai (P Rembali ke) antrian belakang

• Q ideal: 80% proses punya BT < 1 Q

• **WT = Sls Eksek - Waktu Tiba - RT | TAT = WT + BT**

5. Contoh 1. Dalam seleksi (Penerimaan) antrian belakang

- Q ideal: 80% proses punya $BT < 1 Q$

WT = Sls Fksk - Waktu Tiha - BT | TAT = WT + BT

P	BT	Sls Eksk	WT	TAT
A	7	26	19	26
B	5	19	14	19
C	8	28	20	28

• **Q=3; Semua tiba=**

• n proses → masing-masing dapat 1/n kemampuan CPU

• **Guaranteed Scheduling - GS (Preemptive)**

• Proses dengan rasio terkecil dieksekusi lebih dulu

• **JWP = Arrival Time / Banyak Proses (BP)**

• **Rasio R = Burst Time / JWP → Urut: R terkecil**

P	AT	BT	JWP	R	Urutan
P1	2	8	0.5	16	4th
P2	12	15	3	5	3rd
P3	10	10	2.5	4	2nd
P4	8	2	2	1	1st
Eks: P4→P3→P2					BP=4